AI/ML DEEP DIVE

Machine Learning Core Concepts

A Complete Reference for Supervised/Unsupervised/RL, Training Dynamics, Evaluation, Algorithms, and the End-to-End Project Lifecycle

machine-learning · supervised · unsupervised · gradient-descent · evaluation · feature-engineering · interview-prep

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What Is Machine Learning?
2. Types of Machine Learning
3. Features, Labels & Datasets
4. Data Splitting & Cross-Validation
5. Overfitting, Underfitting & the Bias–Variance Tradeoff
6. Regularisation: L1, L2 & Dropout
7. Gradient Descent Variants
8. Loss Functions
9. Evaluation Metrics
10. Feature Engineering, Normalisation & Selection
11. Handling Class Imbalance
12. Common Algorithms Overview
13. The End-to-End ML Project Lifecycle
14. When to Use ML, Rules, or LLMs

What Is Machine Learning?

Machine learning (ML) is a branch of artificial intelligence in which systems learn statistical patterns from data, enabling predictions or decisions on unseen inputs — without being explicitly programmed for every individual case.

TRADITIONAL PROGRAMMING

Input: Rules + Data
Output: Answers

The engineer encodes every decision path explicitly. Brittle when rules change.

MACHINE LEARNING

Input: Data + Answers
Output: A model (learned rules)

The algorithm discovers the mapping function. Adapts as new data arrives.

DEEP LEARNING

A subset of ML using multi-layer neural networks. Learns hierarchical representations from raw inputs (pixels, tokens, waveforms) with minimal manual feature design.

Key Insight

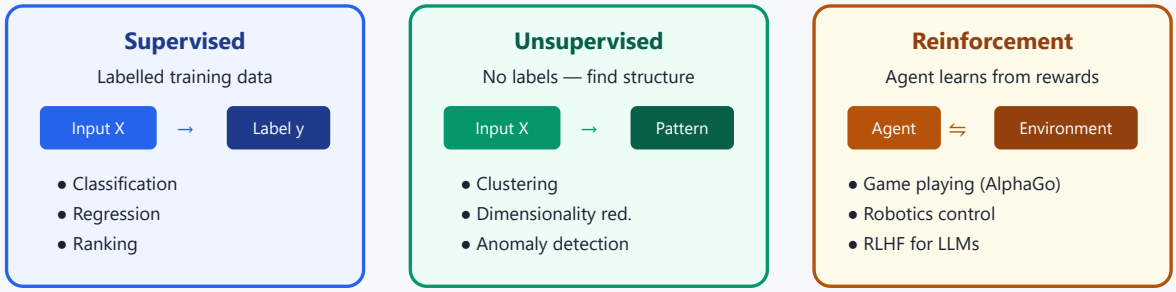
ML excels when rules are too complex, too numerous, or simply unknown — e.g., recognising handwriting, ranking search results, detecting fraud, or personalising recommendations. When rules are simple and stable, classical logic is usually faster and safer.

Mental Model

ML is generalised curve-fitting: given enough labelled examples, an algorithm finds a mathematical function that maps inputs to outputs well enough to generalise. Generalisation — not memorisation — is the whole goal.

Types of Machine Learning

ML algorithms are grouped by the type of feedback signal available during training. Each paradigm suits different problem structures.



The three main ML paradigms differ in whether labels exist (supervised), are absent (unsupervised), or arrive as delayed reward signals (reinforcement).

Semi-Supervised & Self-Supervised

Semi-supervised learning uses a small labelled set plus a large unlabelled set — common when labelling is expensive. **Self-supervised** learning generates its own supervision signal from the data structure itself (e.g., predicting masked tokens in BERT, predicting the next frame in video). Modern LLMs are pre-trained with self-supervision.

> **What is the fundamental difference between supervised and unsupervised learning?**
>
> In supervised learning, every training example has a known target label; the model learns a mapping from inputs to those labels. In unsupervised learning, no labels exist — the algorithm discovers inherent structure (clusters, manifolds, latent factors) in the input distribution itself. The absence of a ground-truth signal makes unsupervised learning harder to evaluate but applicable to far more raw data.

Features, Labels & Datasets

Every ML workflow is built around a structured dataset. Understanding its components is foundational to every other decision.

FEATURES (X)

The input variables used to make a prediction. Also called *predictors*, *covariates*, or *attributes*.

- **Numerical:** continuous (age, price) or discrete (count)
- **Categorical:** nominal (colour, city) or ordinal (low/medium/high)
- **Text / Embeddings:** tokenised, TF-IDF, or dense vectors
- **Images / Audio:** raw pixels or spectrograms
- **Temporal:** time-series lags, rolling statistics

LABELS (Y)

The target variable the model must predict. Shape determines task type:

- **Binary:** 0/1, True/False (binary classification)
- **Multi-class:** integer class index (multi-class)
- **Multi-label:** vector of 0/1 per class
- **Continuous:** real number (regression)
- **Sequence:** token-by-token output (seq2seq)

Dataset Quality Checklist

Property	Why It Matters	How to Check
Size	Insufficient data causes overfitting	Learning curves; validation loss plateau
Representativeness	Distribution shift at inference degrades performance	Compare feature stats: train vs real traffic
Label quality	Noisy labels impose an irreducible error floor	Label consistency inter-annotator agreement
Missing values	Many algorithms cannot handle NaN natively	`df.isnull().sum()`
Duplicates	Inflate evaluation metrics; cause leakage	`df.duplicated().sum()`
Class balance	Imbalanced classes bias naive accuracy	`y.value_counts(normalize=True)`

PYTHON

```python
import pandas as pd
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer(as_frame=True)
df = data.frame          # 569 rows x 31 cols (30 features + target)
X = data.data            # shape (569, 30)
y = data.target          # shape (569,) — binary: 0=malignant, 1=benign

print(X.dtypes.value_counts())   # all float64 here
print(y.value_counts())          # 212 malignant, 357 benign
print(X.isnull().sum().max())    # 0 — no missing values
```

⚠ **Data Leakage**

Never let any information from the test set (or future time steps) influence training — including feature scaling parameters, imputation statistics, or label-derived encodings. Fit all preprocessing only on training data, then transform validation and test sets.

What makes a feature informative, and when does adding more features help vs hurt?

A feature is informative when it has predictive power for the label that is not already captured by existing features. Adding more features helps when they are genuinely predictive and the dataset is large enough to support their estimation. Uninformative or redundant features increase noise, inflate training time, and — in low-data regimes — cause the curse of dimensionality: the volume of feature space grows exponentially while data density shrinks. Regularisation (L1 / dropout) and feature selection methods help prune irrelevant inputs.

Data Splitting & Cross-Validation

Honest evaluation requires that the data used to assess a model was never seen during training or hyperparameter tuning. The three-way split enforces this discipline.

TRAINING SET (~70%)

The model adjusts its weights/parameters to minimise loss on this split.

Risk: over-optimised to this exact sample.

VALIDATION SET (~15%)

Used to tune hyperparameters and select between models. Influences training *indirectly* through your choices.

Risk: repeated use slowly leaks test information.

TEST SET (~15%)

Touched exactly *once* after all decisions are finalised. Gives an unbiased estimate of real-world performance.

Risk: over-reporting if used repeatedly.

✕ **Never Peek at Test**

Evaluating on the test set and then tweaking your model makes the test set a second validation set. Your published metric will be over-optimistic. Reserve the test set for a single final evaluation.

K-Fold Cross-Validation

When data is scarce, k-fold uses all samples for both training and validation. The dataset is partitioned into *k* folds; the model trains on k-1 folds and validates on the remaining fold, rotating until every fold has been the validation set once.



Stratified k-fold preserves the class distribution within each fold — essential for imbalanced datasets. **TimeSeriesSplit** respects temporal order so future data is never used to predict the past.

PYTHON

```python
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier

# Simple 3-way split
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=42
)

# 5-fold cross-validation (no separate val set needed for hyperparameter search)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
scores = cross_val_score(model, X_train, y_train, cv=cv, scoring="roc_auc")
print(f"CV AUC: {scores.mean():.3f} +/- {scores.std():.3f}")
```

> **Why do we need a validation set separate from the test set? Isn't it wasteful?**
>
> The validation set is consumed by the hyperparameter tuning process — every time you check validation performance and adjust your model, you use one "trial" of that data. After many trials the validation score becomes a biased estimate. The held-out test set, never consulted during tuning, provides a genuinely unbiased final estimate. Without this separation you would be unknowingly optimising for the test set and reporting over-inflated numbers. For small datasets the cost in data is managed with k-fold cross-validation for validation, reserving a small test set for the final step.

Overfitting, Underfitting & the Bias–Variance Tradeoff

The central tension in machine learning: a model must be complex enough to capture real patterns, but not so complex that it memorises noise. The bias–variance decomposition formalises this tradeoff.

UNDERFITTING (HIGH BIAS)

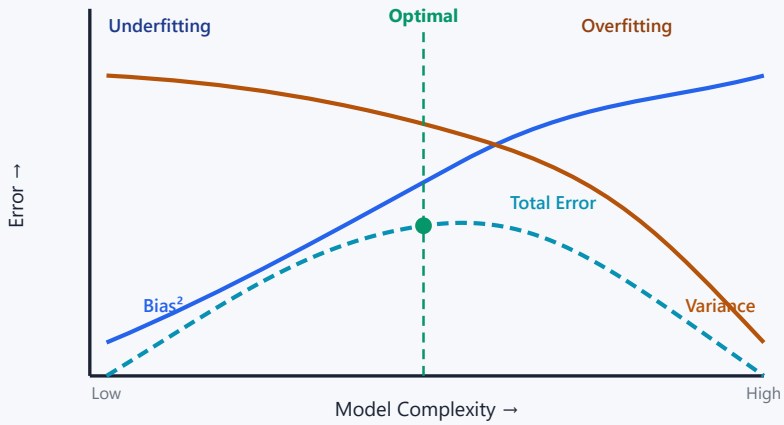
The model is too simple to capture the true relationship. Both training and validation error are high. Symptoms: linear model on non-linear data; insufficient capacity; too much regularisation.

Fix: increase capacity, reduce regularisation, add features.

OVERFITTING (HIGH VARIANCE)

The model memorises training data, including noise. Training error is very low but validation/test error is high. Symptoms: training-validation gap grows as model complexity increases.

Fix: more data, regularisation, early stopping, dropout, simpler model.



Bias–variance tradeoff: as model complexity increases, bias (training error) falls while variance (sensitivity to data noise) rises. The optimal model minimises total expected error at the sweet spot.

The Bias–Variance Decomposition

For a squared-error loss, the expected prediction error decomposes as:

PYTHON

```
# Conceptual decomposition — not runnable alone
# Expected MSE = Bias² + Variance + Irreducible Noise
#
# Bias²  = (E[f̂(x)] − f(x))²   — how far average prediction is from truth
# Variance = E[(f̂(x) − E[f̂(x)])²] — how much prediction varies across datasets
# Noise = inherent randomness in y|x (cannot be reduced by any model)
#
# High bias:     simple model, systematic errors  → underfitting
# High variance: complex model, unstable outputs  → overfitting
```

✦ **Practical Detection**

Plot training and validation loss vs. training epochs (or dataset size). If train loss is low and val loss is high: overfitting. If both are high: underfitting. If both curves are close and both high: irreducible noise or feature insufficiency.

How do you detect and diagnose overfitting in practice?

The canonical signal is a growing gap between training and validation loss during training. Concretely: (1) plot training and validation curves; if val loss starts increasing while train loss keeps decreasing, the model is overfitting. (2) Compare train accuracy vs. val accuracy — a large gap (e.g., 99% vs 78%) is a red flag. (3) Use learning curves: plot both metrics against training set size; overfitting models show high variance (large gap) at small sizes that closes as data grows. Remedies: early stopping, dropout, regularisation (L1/L2), data augmentation, reducing model capacity.

Explain the bias–variance tradeoff in plain language.

Bias is how wrong your model is on average across many different training datasets — a biased model misses the true pattern systematically (e.g., fitting a straight line to curved data). Variance is how much your model changes when trained on different samples of the same data — a high-variance model is too sensitive to the specific training set it saw. As you

> increase model complexity, bias drops (the model can represent more patterns) but variance rises (the model chases noise). The tradeoff says: total error is minimised at an optimal complexity where neither bias nor variance dominates. Regularisation, ensembles, and early stopping shift this sweet spot.

Regularisation: L1, L2 & Dropout

Regularisation adds a complexity penalty to the loss function, discouraging the model from fitting noise. It is the primary tool for reducing variance without sacrificing model capacity.

L2 Regularisation (Ridge / Weight Decay)

Adds the sum of squared weights to the loss. Penalises large weights heavily; shrinks all weights towards zero but rarely to exactly zero. Preferred when all features are expected to contribute.

PYTHON

```python
# L2 loss = MSE + λ * Σ wᵢ²
# λ (alpha) controls regularisation strength; higher α = stronger shrinkage

from sklearn.linear_model import Ridge
ridge = Ridge(alpha=1.0)          # alpha ≡ λ
ridge.fit(X_train, y_train)
print(ridge.coef_)                # all non-zero, but small
```

L1 Regularisation (Lasso)

Adds the sum of absolute weights. Produces *sparse* models — many weights go to exactly zero, performing implicit feature selection. Preferred when you expect only a few features to be relevant.

PYTHON

```python
# L1 loss = MSE + λ * Σ |wᵢ|

from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
n_used = (lasso.coef_ ≠ 0).sum()
print(f"Features used: {n_used} / {X_train.shape[1]}")
```

Elastic Net

A convex combination of L1 and L2 penalties. Retains the grouping effect of L2 (correlated features get similar weights) while still producing sparsity via L1. Controlled by the `l1_ratio` parameter.

Dropout (Neural Networks)

During training, randomly zero out a fraction *p* of neuron activations in each forward pass. This forces the network to learn redundant representations and prevents co-adaptation between neurons. At inference, dropout is disabled and activations

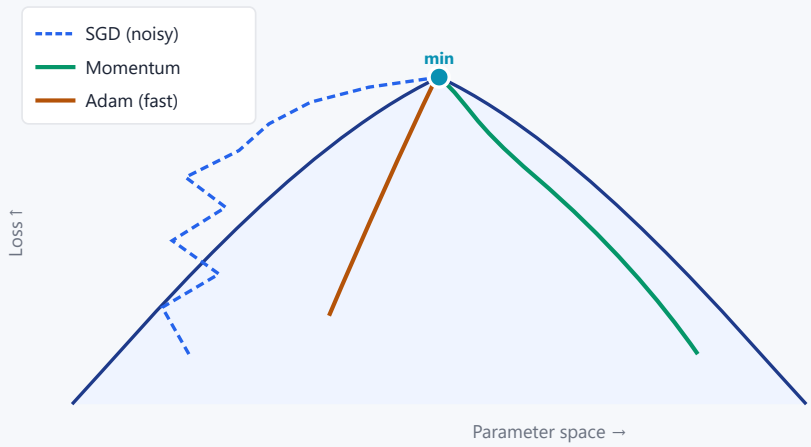
are scaled by *(1−p)*.

Technique	Penalty	Effect on Weights	Feature Selection	Best For
L2 (Ridge)	$\lambda \Sigma w^2$	Uniform shrinkage towards 0	No (all kept)	Many small contributions
L1 (Lasso)	$\lambda \Sigma \lvert w \rvert$	Exact zeros (sparse)	Yes (implicit)	Sparse signal, feature selection
Elastic Net	L1 + L2	Sparse + grouped	Yes	Correlated features
Dropout	Stochastic masking	Implicit weight sharing	No	Neural networks
Early Stopping	Halt training	Reduces effective capacity	No	Deep learning, GBMs

> **When should you prefer L1 over L2 regularisation?**
>
> Choose L1 (Lasso) when you believe the true signal is sparse — that only a small subset of features actually predict the target. Lasso drives irrelevant feature weights to exactly zero, which both improves generalisation and makes the model interpretable. Choose L2 (Ridge) when you expect most features to contribute somewhat — it distributes shrinkage smoothly and handles correlated features better (Lasso arbitrarily picks one from a correlated group). In practice, start with L2 unless you have a specific reason for sparsity; use Elastic Net when in doubt with many correlated predictors.

Gradient Descent Variants

Most ML and all deep learning models are trained by minimising a loss function via gradient descent: iteratively moving parameters in the direction that reduces loss most steeply.



Loss surface (cross-section). SGD zigzags noisily; momentum accumulates velocity for smoother descent; Adam adapts per-parameter learning rates for fast, direct convergence.

BATCH GD

Computes gradient over entire dataset. Slow per-step; very smooth updates. Impractical for large datasets.

SGD (MINI-BATCH)

Gradient over a random mini-batch (32–512 samples). Noisy updates escape local minima; GPU-parallelisable. The standard for deep learning.

MOMENTUM

Adds exponentially-decaying moving average of past gradients ($\beta \approx 0.9$). Dampens oscillations; accelerates along consistent directions.

RMSPROP

Divides learning rate by an exponentially-weighted RMS of recent gradients. Adapts the learning rate per-parameter. Works well for non-stationary objectives (RNNs).

ADAM (ADAPTIVE MOMENT ESTIMATION)

Combines momentum (first moment) and RMSProp (second moment). Includes bias-correction in early steps. Usually the default choice. `lr=1e-3, β₁=0.9, β₂=0.999, ε=1e-8`.

PYTHON

```python
from sklearn.linear_model import SGDClassifier

# SGD with momentum (via sklearn interface)
sgd = SGDClassifier(
    loss="log_loss",        # logistic regression
    penalty="l2",
    alpha=1e-4,             # regularisation strength
    learning_rate="optimal",
    max_iter=1000,
    random_state=42,
)
sgd.fit(X_train, y_train)

# For deep learning (PyTorch):
# optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
# optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

Learning Rate

The most important hyperparameter. Too large: loss diverges (gradient explosion). Too small: training stalls. Use a learning rate finder or cosine annealing scheduler. A common starting point: Adam with lr=1e-3 for deep models, SGD with lr=0.01-0.1 for shallow models.

When would you choose SGD with momentum over Adam, and vice versa?

Adam is a good default for most deep learning scenarios — it converges fast, is robust to learning rate choice, and handles sparse gradients well (NLP, embeddings). However, some research shows that SGD with momentum, when carefully tuned, generalises slightly better for image classification (the "generalisation gap" of adaptive optimisers). Choose SGD + momentum when: you have time to tune the learning rate and schedule, you care about maximum test accuracy, or you are training on well-understood architectures (CNNs for vision). Choose Adam when: training is unstable, gradients are sparse, you want fast convergence with minimal tuning, or you are experimenting/prototyping.

Loss Functions

The loss function quantifies how wrong the model's predictions are. Training minimises this function. Choosing the right loss is as important as choosing the right architecture.

Regression Losses

Loss	Formula	Outlier Sensitivity	When to Use
MSE (L2)	mean((y − ŷ)²)	High — squares errors	Default regression; Gaussian noise assumption
MAE (L1)	mean(|y − ŷ|)	Low — linear penalty	Outlier-heavy targets; median regression
Huber	L2 if |e|<δ else L1·δ − δ²/2	Moderate — hybrid	Best of both; δ is a tunable threshold
MAPE	mean(|y − ŷ| / |y|) × 100	High near zero	Relative error matters; avoid when y≈0

Classification Losses

Loss	Use Case	Notes
Binary Cross-Entropy	Binary classification	Assumes sigmoid output; maximises log-likelihood
Categorical Cross-Entropy	Multi-class (one-hot targets)	Assumes softmax output; most common for classification
Sparse Categorical CE	Multi-class (integer labels)	Identical mathematics; avoids one-hot memory
Hinge Loss	SVM / margin classifiers	max(0, 1 − y·ŷ); promotes max-margin separation
Focal Loss	Severe class imbalance	Downweights easy examples; α·(1−p_t)^γ·CE
KL Divergence	Distribution matching, VAEs	Asymmetric; measures P from Q difference

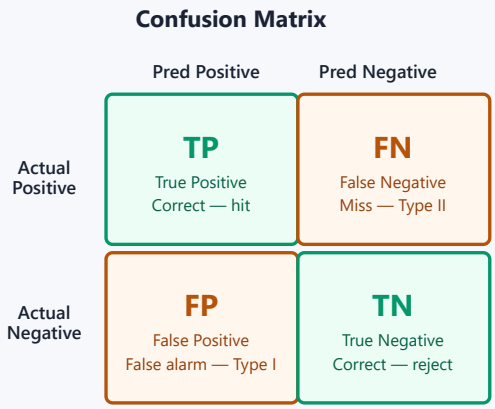
✦ **Cross-Entropy Intuition**

Binary cross-entropy is −log(p) when the true label is 1 and −log(1−p) when it is 0. As the model's predicted probability for the correct class approaches 1, the loss approaches 0. Large penalties are assigned to confident wrong predictions (predicting p=0.01 when y=1 yields loss ≈ 4.6).

Why cross-entropy for classification, not MSE?

Evaluation Metrics

A model trained on one metric may perform poorly on another. Selecting the right metric requires understanding your task, class distribution, and the relative cost of different errors.



Confusion matrix: TP and TN are correct predictions; FP (false alarm, Type I) and FN (miss, Type II) are the two error types. The relative cost of FP vs FN determines which metric to optimise.

Metrics Cheat-Sheet

Metric	Formula	Range	When to Use	Pitfall
Accuracy	(TP+TN) / N	0–1	Balanced classes, simple baseline	Misleading on imbalanced data (99% acc on 1% positive)
Precision	TP / (TP+FP)	0–1	FP is costly (spam filter, fraud alert)	Ignores FN; can be gamed by predicting rarely
Recall (Sensitivity)	TP / (TP+FN)	0–1	FN is costly (disease screening, safety)	Ignores FP; can be gamed by predicting always
F1 Score	2·P·R / (P+R)	0–1	Balanced FP/FN tradeoff, imbalanced classes	Ignores TN; not suitable when TN matters
F-beta	$(1+\beta^2)\cdot P\cdot R / (\beta^2\cdot P+R)$	0–1	$\beta>1$ weights recall; $\beta<1$ weights precision	Requires explicit cost reasoning to choose β
ROC-AUC	Area under TPR vs FPR curve	0.5–1	Comparing classifiers; threshold-agnostic	Optimistic on severely imbalanced data
PR-AUC	Area under Precision-Recall curve	0–1	Imbalanced datasets; rare positive class	Harder to interpret; baseline = prevalence
MSE	$\text{mean}((y-\hat{y})^2)$	0–∞	Regression training objective	Penalises large errors quadratically; sensitive to outliers
RMSE	√MSE	0–∞	Regression reporting (same units as y)	Still outlier-sensitive; dominated by largest errors
MAE	$\text{mean}(\lvert y-\hat{y}\rvert)$	0–∞	Robust regression reporting	Gradient discontinuous at 0; less common as loss
R^2	1 − SS_res/SS_tot	−∞–1	Explained variance fraction	Can increase by adding irrelevant features (use adj-R^2)

PYTHON

```python
from sklearn.metrics import (
    classification_report, roc_auc_score, average_precision_score,
    confusion_matrix, mean_squared_error, mean_absolute_error, r2_score
)
import numpy as np

# Classification
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print(f"ROC-AUC : {roc_auc_score(y_test, y_proba):.4f}")
print(f"PR-AUC  : {average_precision_score(y_test, y_proba):.4f}")

# Regression
rmse = np.sqrt(mean_squared_error(y_test, y_pred_reg))
mae  = mean_absolute_error(y_test, y_pred_reg)
r2   = r2_score(y_test, y_pred_reg)
print(f"RMSE={rmse:.3f}  MAE={mae:.3f}  R²={r2:.3f}")
```

When should you optimise for precision vs recall?

Choose recall (minimise false negatives) when the cost of missing a positive is high: cancer detection (missing a tumour is dangerous), fraud detection (letting fraud pass is expensive), safety-critical alerts. Choose precision (minimise false positives) when the cost of a false alarm is high: spam filtering (a legitimate email in spam is annoying), surgical intervention (unnecessary treatment has risk), ad targeting (irrelevant ads waste budget). The F-beta score lets you quantify this tradeoff: $\beta > 1$ rewards recall more, $\beta < 1$ rewards precision more. In practice, always report both and plot the PR curve to understand the tradeoff at every threshold.

What does ROC-AUC measure and when is it preferable to accuracy?

ROC-AUC is the probability that a randomly chosen positive example scores higher than a randomly chosen negative example. It measures the ranking quality of the model across all possible decision thresholds (0 to 1). AUC = 0.5 is random; AUC = 1.0 is perfect. It is preferable to accuracy when: (1) classes are imbalanced — accuracy can be 99% by predicting the majority class always, while AUC reveals poor discrimination; (2) the final decision threshold is not yet fixed; (3) you want to compare two classifiers irrespective of threshold. For severely imbalanced data (e.g. 0.1% positives), PR-AUC is even more informative because ROC-AUC can be optimistic about performance on the rare class.

Feature Engineering, Normalisation & Selection

Raw data is rarely in the form an algorithm needs. Feature engineering transforms, creates, and selects the representation that lets a model learn most efficiently.

Numerical Scaling

STANDARDISATION (Z-SCORE)

$x' = (x - \mu) / \sigma \rightarrow$ zero mean, unit variance.

Use when: algorithm assumes Gaussian features or uses distance/gradient (SVM, KNN, neural nets, PCA, logistic regression).

Not required for: tree-based models (decision trees, random forests, GBMs).

MIN-MAX NORMALISATION

$x' = (x - x_min) / (x_max - x_min) \rightarrow$ maps to [0, 1].

Use when: you need bounded inputs (image pixel values, neural net inputs with sigmoid output).

Caution: sensitive to outliers — a single extreme value compresses all others near 0.

Categorical Encoding

Method	Output	When to Use
One-Hot Encoding	Binary vector (1 per category)	Nominal categories, linear models, NNs; avoid if cardinality > 50
Ordinal Encoding	Integer 0,1,2…	Ordered categories (low/medium/high); tree-based models
Target Encoding	Mean of label per category	High-cardinality categories; use cross-fold to avoid leakage
Frequency Encoding	Count of occurrence	High-cardinality; tree models; no leakage risk
Embedding	Dense low-dim vector	Neural networks; very high cardinality (user IDs, words)

Feature Creation

Domain knowledge is the most valuable source of new features:

- **Polynomial / interaction features:** $x_1 \cdot x_2$, x_1^2, useful for linear models
- **Log / sqrt transforms:** compress right-skewed distributions; stabilise variance
- **Aggregations:** user-level statistics (mean purchase value, rolling 7-day activity)
- **Date decomposition:** day-of-week, hour, quarter, is_weekend
- **Text:** TF-IDF, BERT embeddings, sentence length, sentiment score

Feature Selection

PYTHON

```python
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.feature_selection import SelectKBest, f_classif, RFE
from sklearn.decomposition import PCA

# Standardise numerical features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)   # fit only on train!
X_test_scaled  = scaler.transform(X_test)         # apply same transform

# Univariate filter (ANOVA F-test): keep top 10 features
selector = SelectKBest(score_func=f_classif, k=10)
X_top10 = selector.fit_transform(X_train_scaled, y_train)
print("Selected features:", selector.get_support(indices=True))

# Recursive Feature Elimination (wrapper method)
from sklearn.linear_model import LogisticRegression
rfe = RFE(estimator=LogisticRegression(), n_features_to_select=10)
rfe.fit(X_train_scaled, y_train)
print("RFE kept:", X_train.columns[rfe.support_].tolist())

# PCA for dimensionality reduction (not feature selection)
pca = PCA(n_components=0.95)   # keep 95% of variance
X_pca = pca.fit_transform(X_train_scaled)
print(f"Components to retain 95% var: {pca.n_components_}")
```

When is feature normalisation required and when can you skip it?

Normalisation (standardisation or min-max) is required for algorithms that are sensitive to feature scale: gradient-based methods (logistic regression, neural nets, SVMs — the gradient step size depends on the magnitude of features), distance-based methods (KNN, K-means — Euclidean distance is dominated by large-scale features), and PCA (variance is scale-dependent). Tree-based algorithms (decision trees, random forests, XGBoost) are invariant to monotonic feature transforms — they only care about the relative order of values, not their absolute scale, so normalisation has no effect. As a rule of thumb: any algorithm with a cost function that sums across features, or one that uses distances, needs scaling.

Handling Class Imbalance

Real-world datasets are rarely balanced. Fraud (0.1%), rare diseases, and defect detection all exhibit severe imbalance. Naive training produces models that classify everything as the majority class and report spuriously high accuracy.

⚠ **Imbalance Warning**

Accuracy is useless on imbalanced data. A dataset with 99% negatives achieves 99% accuracy by always predicting negative. Use F1, PR-AUC, or ROC-AUC as primary metrics, and always examine per-class performance in the classification report.

Strategy 1: Resampling

OVERSAMPLING (SMOTE)

Synthetic Minority Oversampling Technique generates new minority examples by interpolating between existing ones in feature space. Better than random duplication — avoids exact overfitting to existing samples.

Risk: can generate unrealistic examples if feature space is sparse.

UNDERSAMPLING

Remove majority class examples randomly (or by clustering with Tomek Links). Simple; speeds up training. Risk: discards potentially informative examples.

Rule of thumb: undersample when dataset is large; oversample when small.

Strategy 2: Class Weights

Most sklearn estimators accept `class_weight="balanced"`, which sets weight inversely proportional to class frequency. This penalises misclassification of the minority class more heavily during training — no data modification required.

Strategy 3: Threshold Tuning

A default probability threshold of 0.5 is arbitrary. By lowering the threshold (e.g., predict positive if p > 0.3) you increase recall at the cost of precision. Use the PR curve to select a threshold matching your deployment needs.

PYTHON

```python
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
import numpy as np

# Strategy 1: SMOTE oversampling
sm = SMOTE(random_state=42, k_neighbors=5)
X_res, y_res = sm.fit_resample(X_train, y_train)
print(f"Before: {np.bincount(y_train)}")
print(f"After:  {np.bincount(y_res)}")

# Strategy 2: class_weight
clf = LogisticRegression(class_weight="balanced", max_iter=1000)
clf.fit(X_train, y_train)
print(classification_report(y_test, clf.predict(X_test)))

# Strategy 3: threshold tuning
proba = clf.predict_proba(X_test)[:, 1]
threshold = 0.30
y_pred_thresh = (proba ≥ threshold).astype(int)
print(classification_report(y_test, y_pred_thresh))
```

> **You have a dataset with 5% positive class. What techniques and metrics would you use?**
>
> First, never use accuracy — report precision, recall, F1 (or F-beta), and PR-AUC. For training: apply class_weight="balanced" as a first step (zero cost, often surprisingly effective). If performance is still poor, apply SMOTE on the training fold only (never on val/test). Consider using ensemble methods (random forest, XGBoost with scale_pos_weight parameter) as they

> are naturally more robust to imbalance. After training, tune the decision threshold using the precision-recall curve to match your business cost tradeoff (e.g., detecting 90% of positives at the cost of 50% precision may be acceptable for fraud screening but not for surgery recommendations). Monitor for concept drift in production — the positive rate may shift over time.

Common Algorithms Overview

No single algorithm wins on all problems. Understanding the strengths, assumptions, and failure modes of each helps you choose intelligently and explain decisions to stakeholders.

Algorithm	Type	Task	Key Params	Strengths	Weaknesses	Best For
Linear Regression	Supervised	Regression	alpha (L2/L1)	Fast, interpretable, analytical solution	Assumes linearity & homoscedasticity	Continuous target with linear signal; baseline
Logistic Regression	Supervised	Classification	C (1/λ), penalty	Probabilistic output, fast, interpretable	Linear decision boundary; poor on non-linear data	Binary/multi-class with interpretability; NLP baselines
Decision Tree	Supervised	Both	max_depth, min_samples_split	Interpretable (visualisable), handles mixed types, non-linear	High variance, overfits without pruning	Prototypes, rule extraction, explainability
Random Forest	Supervised	Both	n_estimators, max_features	Robust, implicit feature importance, handles missing values	Large memory, slower inference, black box	Tabular data, competitions, feature selection
Gradient Boosting (XGBoost/LightGBM)	Supervised	Both	n_estimators, learning_rate, max_depth	State-of-the-art on tabular; handles imbalance	Many hyperparameters; slower to train than RF	Kaggle-style structured data; production tabular
SVM	Supervised	Both	C, kernel, gamma	Effective in high-D, kernel trick for non-linearity	Slow on large datasets ($O(n^2)$); needs scaling	Text classification, small/medium datasets, high-D
KNN	Supervised	Both	n_neighbors, metric	Non-parametric, intuitive, no training phase	Slow inference O(nd), poor scaling, needs scaling	Recommendation, anomaly detection, small datasets
Naïve Bayes	Supervised	Classification	var_smoothing (GNB)	Extremely fast, works well on text, handles missing	Strong independence assumption rarely holds exactly	Text classification, spam filtering, NLP baseline
K-Means	Unsupervised	Clustering	n_clusters, init, n_init	Simple, fast, scalable	Must choose K; assumes spherical	Customer segmentation, image

					clusters; sensitive to init	compression, pre-processing
PCA	Unsupervised	Dim. reduction	n_components	Removes correlated features, speeds up downstream training	Components are linear combinations; interpretability lost	Visualisation, de-noising, pre-processing before ML

When does random forest outperform a single decision tree?

PYTHON

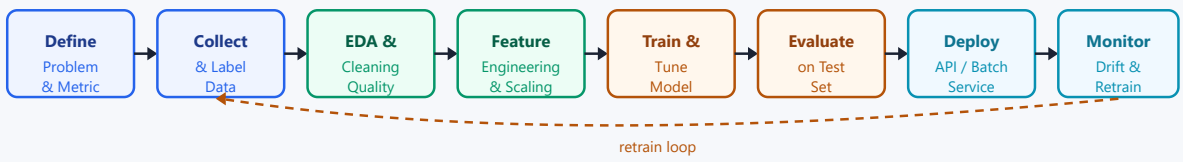
```python
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=200, random_rate=0.1),
    "SVM (RBF)": SVC(probability=True, kernel="rbf"),
    "KNN": KNeighborsClassifier(n_neighbors=5),
}

for name, clf in models.items():
    clf.fit(X_train_scaled, y_train)
    auc = roc_auc_score(y_test, clf.predict_proba(X_test_scaled)[:, 1])
    print(f"{name:25s}  AUC: {auc:.4f}")
```

The End-to-End ML Project Lifecycle

Successful ML projects follow a disciplined lifecycle. Model training is only a fraction of the total effort — problem framing, data work, and deployment typically dominate.



ML project lifecycle. The retrain loop (dashed orange) is often forgotten: models degrade as real-world distributions drift. Monitoring data quality and prediction quality in production is as important as initial training.

1. Define Problem & Success Metric

2. Data Collection & Labelling

3. EDA & Data Cleaning

4-5. Feature Engineering & Model Training

6-8. Evaluate, Deploy & Monitor

PYTHON

```python
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

# Build a pipeline to prevent leakage
numeric_features = ["age", "income", "days_since_last_purchase"]
categorical_features = ["country", "device_type"]

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
])

pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier",   RandomForestClassifier(random_state=42)),
])

# Hyperparameter tuning on train+val (cross-validated)
param_grid = {
    "classifier__n_estimators": [100, 200],
    "classifier__max_depth": [None, 10, 20],
}
search = GridSearchCV(pipe, param_grid, cv=5, scoring="roc_auc", n_jobs=-1)
search.fit(X_train, y_train)

print(f"Best CV AUC: {search.best_score_:.4f}")
print(f"Best params: {search.best_params_}")

# Final evaluation — only here do we touch the test set
from sklearn.metrics import roc_auc_score
final_auc = roc_auc_score(y_test, search.predict_proba(X_test)[:, 1])
print(f"Test AUC:   {final_auc:.4f}")
```

When to Use ML, Rules, or LLMs

Not every problem benefits from machine learning. Choosing the wrong paradigm wastes engineering effort and introduces unnecessary risk. This section provides a practical decision framework.

USE RULE-BASED LOGIC WHEN

- Rules are known, stable, and small in number
- Full transparency and auditability is required
- Data is too scarce to train reliably
- Edge cases are safety-critical and must be enumerated
- Regulatory compliance requires human-readable logic

Examples: input validation, access control, pricing formulas, deterministic routing.

USE TRADITIONAL ML WHEN

- Labelled training data is available at scale
- The pattern is too complex for manual rules
- Input is structured/tabular
- Low-latency inference is required
- Model explainability matters to stakeholders

Examples: fraud detection, churn prediction, demand forecasting, image classification.

USE AN LLM WHEN

- Task requires open-ended language generation
- Instruction-following from natural language prompts is needed
- Labelled training data is absent but a capable base model exists
- The task is too diverse for a narrow ML model
- Few-shot adaptation is sufficient

Examples: summarisation, Q&A, code generation, classification via prompting, agentic tasks.

✦ **Hybrid Approach**

Production systems often combine all three: rules enforce hard constraints (data validation, safety guardrails), traditional ML handles structured prediction tasks (scoring, ranking), and an LLM handles open-ended language tasks. Each layer uses the simplest appropriate tool.

Dimension	Rules	Traditional ML	LLM
Training data needed	None	Large labelled set	None (few-shot) or large (fine-tune)
Inference latency	Microseconds	Milliseconds	Hundreds of ms to seconds
Explainability	Full	Medium (tree-based) – Low (NN)	Low (prompt-level reasoning)
Maintenance	High (manual updates)	Medium (retrain on drift)	Low (prompt updates)
Cost per call	~Zero	Low (hosted model)	Higher (API tokens)
Handles novel inputs	No (breaks on edge cases)	Extrapolates within distribution	Strong generalisation
Input type	Structured values	Structured / engineered features	Natural language, multimodal

Best Practices Summary

DATA HYGIENE

- Never fit preprocessors on validation/test data
- Always stratify splits when classes are imbalanced
- Use Pipelines to bundle preprocessing with the model
- Check for data leakage before reporting results
- Track dataset versions alongside model versions

MODELLING

- Start with the simplest possible baseline
- Use cross-validation for small datasets; hold-out for large
- Fix random seeds for reproducibility
- Log every experiment (params, metrics, artifacts)
- Evaluate on test set exactly once at the end

METRICS

- Choose your metric before training, not after
- Report multiple metrics; never just accuracy
- Disaggregate metrics by subgroup for fairness
- Establish a human or rules-based baseline to compare against

PRODUCTION

- Monitor data drift and prediction distribution in prod
- Version your models and roll back gracefully
- Set alerting thresholds on business KPIs, not just ML metrics
- Plan for retraining frequency before you deploy

✓ **Key Takeaway**

Most ML failures are not model failures — they are data quality failures, metric misalignment failures, or deployment failures. Invest heavily in the non-model parts of the pipeline: data validation, robust feature engineering, honest evaluation, and production monitoring.

A stakeholder asks why you chose a random forest over a neural network for a tabular churn prediction task. What do you say?

For tabular data at typical enterprise scales (tens of thousands to a few million rows), tree-based ensembles (random forest, gradient boosting) consistently match or outperform deep networks. They require less data, need little preprocessing (no normalisation required, handle mixed feature types naturally), train in seconds rather than hours, and provide interpretable feature importances out of the box. Neural networks excel on raw unstructured data (images, text, audio) where representation learning is valuable. For a churn model with structured CRM features, a gradient-boosted tree gives better performance for less engineering cost, and a stakeholder can understand "recency of last purchase was the top feature" in a business meeting.

When would you recommend an LLM over a fine-tuned classification model for a text classification task?

Use an LLM (via prompting or few-shot) when: (1) labelled data is scarce or expensive to produce — an LLM can classify with zero or a handful of examples; (2) the label taxonomy is dynamic or evolving — you can update a prompt in minutes vs retraining a model; (3) the classification requires reasoning or multi-hop context that a bag-of-words or encoder-only model struggles with; (4) the task scope is too diverse for a single narrow model. Use a fine-tuned classifier when: you have thousands of labelled examples, latency must be under 100ms, API costs at scale are prohibitive, or you need a self-hosted solution with no external data dependency. A common pattern: use an LLM to label thousands of examples cheaply, then fine-tune a small classifier on those labels for production serving.